ISPilot Architecture Reset: From Power Automate to Teams SDK

Date: 2026-09-01
Status: Architecture Updated
Previous Approach: GCP IAM + Power Automate
New Approach: Microsoft 365 Agents SDK + Teams Bridge

Why We Changed

Initially, we considered using **Power Automate** to call ispilot-api directly via GCP IAM bearer tokens. This approach had several problems:

1. **Token Type Mismatch**: Power Automate generates Microsoft Bearer tokens, not Google identity tokens
    - ispilot-api expects Google Cloud identity tokens from sa-tot-osa
    - Token formats and validation chains don't align
2. **Authentication Complexity**: Trying to use GCP IAM for a Teams client adds unnecessary layers
    - Power Automate doesn't have native GCP Workload Identity support
    - Required manual token generation and rotation logic
    - Cross-cloud authentication (Azure→Google) is not idiomatic
3. **No Native Teams Integration**: Power Automate is designed for workflow automation, not interactive bot conversations
    - Doesn't provide session context
    - Doesn't handle Teams message lifecycle
    - Required reimplementation of Teams protocol handling

New Architecture: Microsoft 365 Agents SDK

The **Microsoft 365 Agents SDK** provides a native solution:

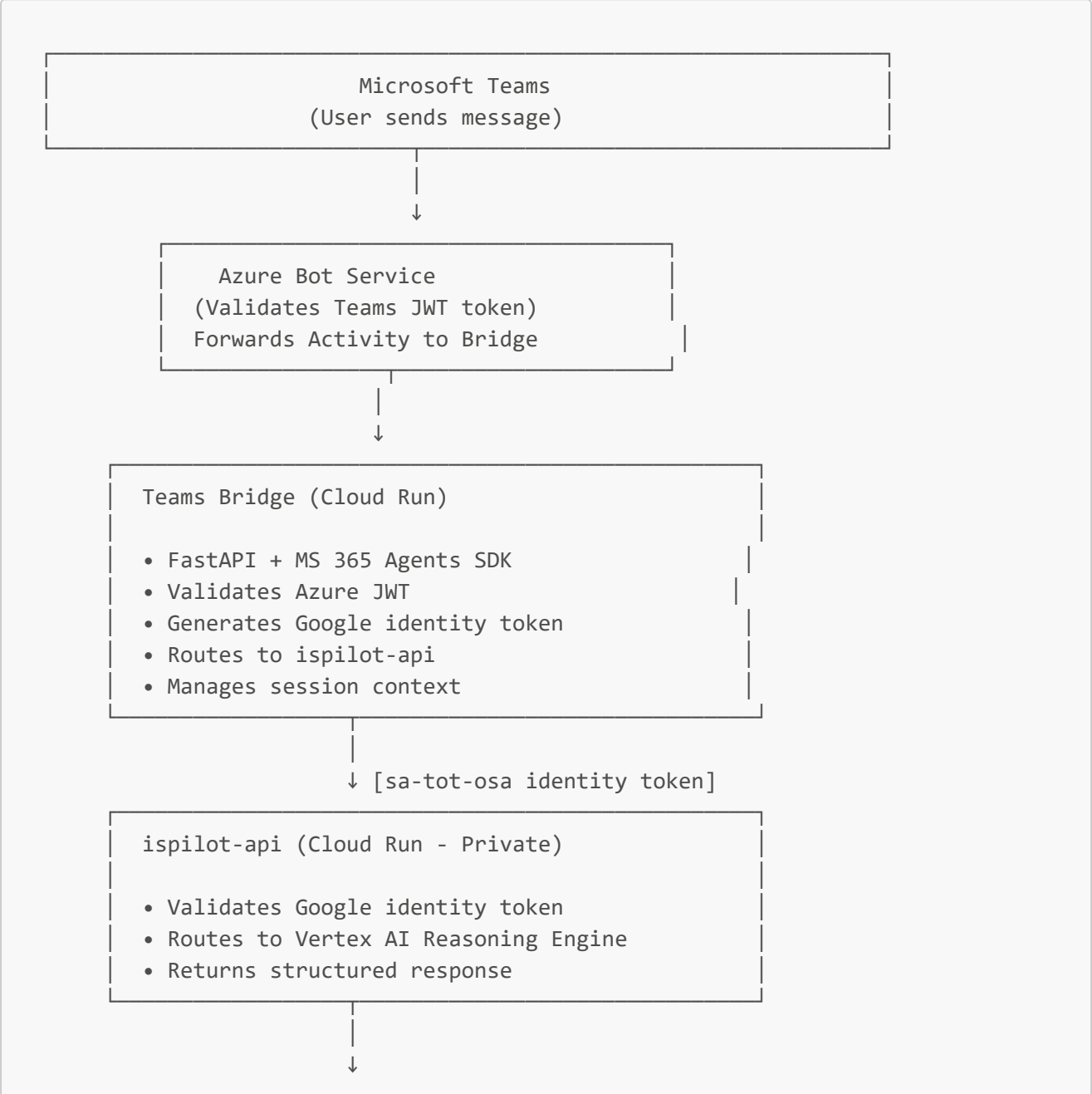
What Changed

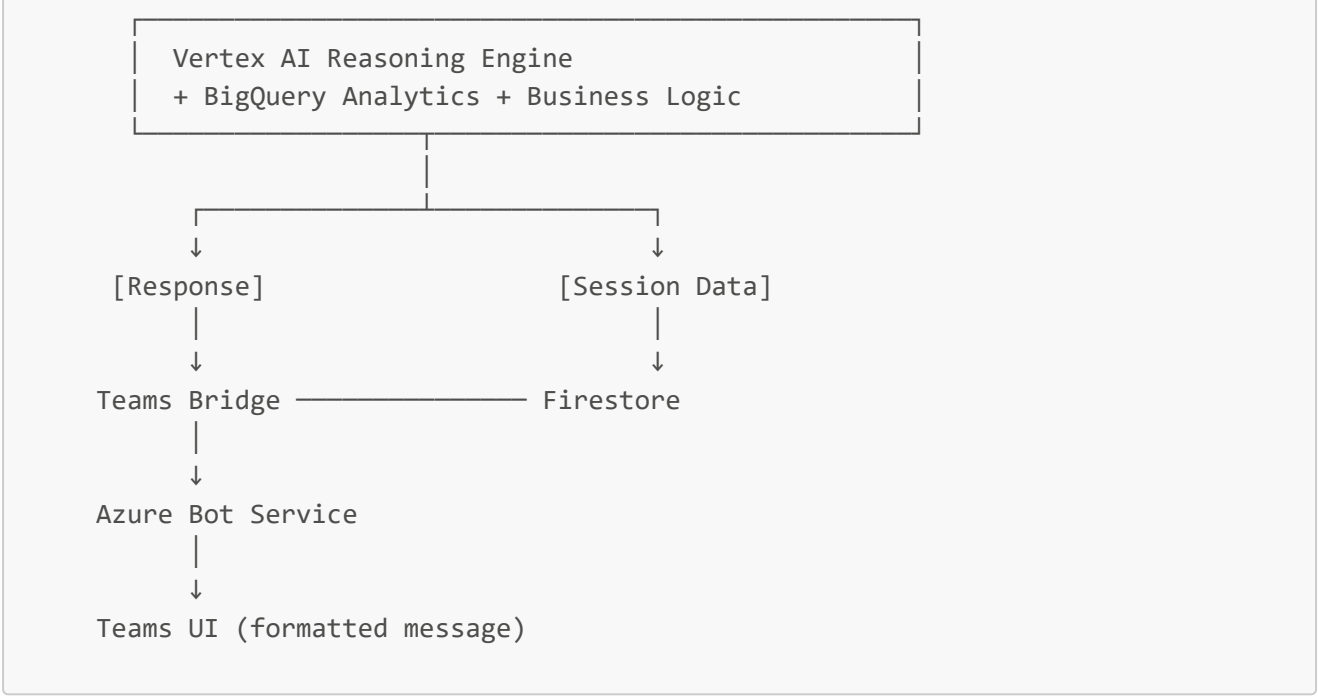
```
❌ OLD: Teams → Power Automate → (GCP Bearer) → ispilot-api
☑ NEW: Teams → Teams Bridge (SDK) → (Google Identity) → ispilot-api
```

Key Differences

Aspect	Old Approach	New Approach
Teams Integration	Custom Power Automate flow	MS 365 Agents SDK (native)
JWT Validation	Manual in Power Automate	Built into Agents SDK
Service Account	sa-teams-bridge (Azure)	sa-tot-osa (Google)
Token Type	Microsoft Bearer	Google Identity
Bot Registration	Not needed	Teams Developer Portal
Session Context	Lost between calls	Preserved in Bridge
Error Handling	Flow-based (clunky)	Python exception handling
Scalability	Limited to Power Automate quotas	Full Cloud Run scaling

Architecture Diagram





What This Means for Each Component

ispilot-api (No Changes)

- ☑ Continues to validate Google identity tokens
- ☑ Continues to work exactly as deployed
- ☑ Now receives requests only from Teams Bridge (or direct GCP clients)
- ☑ No Power Automate integration needed

Teams Bridge (New)

- 🆕 New Cloud Run service using MS 365 Agents SDK
- 🆕 Handles Teams ↔ Azure authentication
- 🆕 Acts as proxy for ispilot-api calls
- 🆕 Maintains session context between messages

Root Agent Project (No Changes)

- ☑ Continues to operate as business logic layer
- ☑ ispilot-api exposes it via REST
- ☑ Teams Bridge consumes the REST API

Azure / Teams Resources (New Setup)

- 🆕 Register bot in Azure AD
- 🆕 Create Teams app manifest
- 🆕 Publish to Teams catalog

Migration Path

Phase 1: Keep Current State (Week 1)

- ☑ ispilot-api remains production stable
- ☑ Continue with current direct API validation
- ☑ No immediate changes needed

Phase 2: Build Teams Bridge (Week 2-3)

- 🔄 Create `teams_bot_bridge/` service
- 🔄 Implement MS 365 SDK handlers
- 🔄 Deploy to Cloud Run
- 🔄 Test with manual Teams bot registration

Phase 3: Register & Publish (Week 4)

- 🔄 Complete Azure bot registration
- 🔄 Publish Teams app manifest
- 🔄 Roll out to Teams users

Phase 4: Validation & Monitoring (Ongoing)

- 🔄 Business scenario testing
- 🔄 Performance monitoring
- 🔄 Error rate tracking
- 🔄 User feedback

Why This Approach is Better

☑ Standards Compliance

- Uses official Microsoft SDK (supported, well-documented)
- Follows Teams protocol correctly
- Azure-native authentication

☑ Simpler Token Management

- Bridge handles Azure JWT validation (built-in)
- Bridge generates Google tokens only when needed
- No cross-cloud token translation

☑ Better Session Handling

- Teams activity context is preserved
- Conversation ID maps directly to session
- Multi-turn conversations work naturally

☑ Production Ready

- Microsoft SDK is used by enterprise Teams apps

- Better error handling and logging
- Scaling is handled by Cloud Run

☑ Maintainability

- Centralized bot logic in Bridge
- ispilot-api remains focused on business logic
- Clear separation of concerns

Power Automate vs Teams SDK: When to Use Each

Use Case	Power Automate	Teams SDK
Custom Teams Bot	❌ Complex	☑ Native
Workflow Automation	☑ Ideal	❌ Overkill
Multi-turn Chat	❌ Clunky	☑ Natural
Real-time Streaming	❌ No	☑ Yes
Complex Logic	❌ Flow-based	☑ Python code
Cross-cloud auth	❌ Hard	☑ Manageable

Conclusion: For an AI agent in Teams, Microsoft 365 Agents SDK is the right tool.

Rollback Plan (If Needed)

If Teams SDK approach encounters blockers:

1. **Revert to Direct API Calls**:
    - Keep ispilot-api as-is
    - Users call API directly (Copilot plugin, web UI, etc.)
    - No Teams integration (acceptable fallback)
2. **Lightweight Alternative**:
    - Use Azure Functions instead of Cloud Run
    - Simpler Azure-native deployment
    - Still uses Agents SDK
3. **Power Automate (Last Resort)**:
    - If absolutely required for organization
    - Would need ispilot-api to accept Microsoft bearer tokens
    - Adds complexity but technically feasible

References

- MS 365 Agents SDK Documentation
- FastAPI + Teams Integration
- Google Cloud Identity Tokens
- Workload Identity Federation

Document History

Date	Change
2026-09-01	Created: Architecture reset from Power Automate to Teams SDK